

FPT University
HO CHI MINH CITY CAMPUS



FPT UNIVERSITY

Linux and Open Source Platform

Lab 2:

Writing Device Drivers & File System Management

Name	Roll number	Subject
Hoàng Ngọc Gia Bảo	SE203238	LOS201
Trần Hồ Khánh Tân	SE203740	LOS201
Nguyễn Trọng Hùng	SE204352	LOS201
Nguyễn Quế Vũ	SE204290	LOS201
Nguyễn Việt Anh	SE204364	LOS201
Hồ Phan Anh Tuấn	SE203437	LOS201

Lecturer: Phạm Thế Vinh

Ho Chi Minh City, Sep 25, 2026

SECTION 1: Hardware & Software Specifications

- **Host Operating System:** Ubuntu Linux x86_64 (Host Kernel: 7.0.0-31-generic)
- **Target Architecture:** ARMv7-A (Cortex-A9, QEMU vexpress-a9 machine, 2 VCPUs SMP, 512MB RAM)
- **Target Linux Kernel:** Linux Kernel version 5.15.0 LTS
- **Cross-Compilation Toolchain:** arm-linux-gnueabi-hf-gcc version 15.2.0 (Ubuntu 15.2.0-16ubuntu1)
- **Emulation Engine:** QEMU Emulator (qemu-system-arm)
- **Core Utilities:** BusyBox v1.35.0 (Static Build), mtd-utils (flash_erase, mkfs.jffs2, nandwrite, nanddump, mtdinfo), nandsim flash emulator

SECTION 2: DRIVER ARCHITECTURE & DATA FLOW DESIGN

2.1. System & Driver Architecture Diagram

The diagram below illustrates the complete architecture of the lab2_driver character device driver, detailing the interaction across User Space, Virtual File System (VFS), Kernel Space internal buffers, synchronization primitives, and the virtual export interfaces (procfs and sysfs).

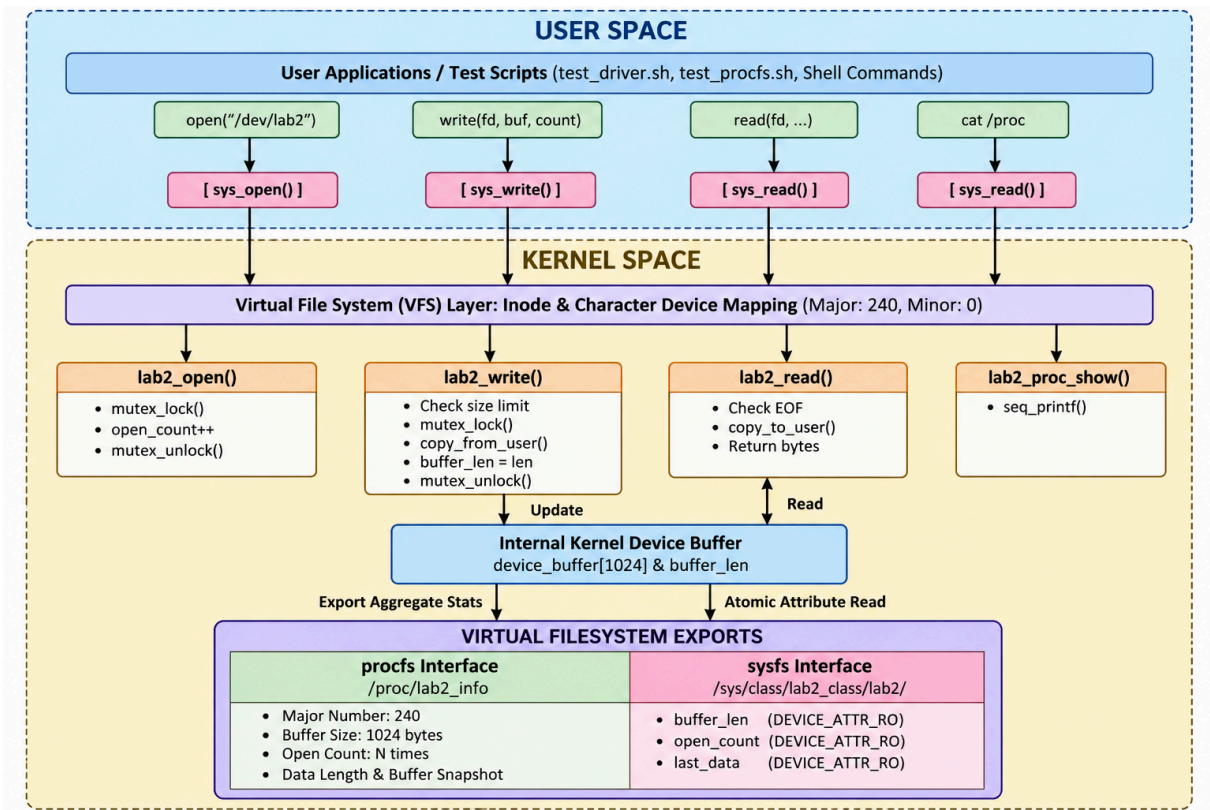


Figure 2.1: Architectural design and communication channels of the character device driver.

2.2. Architectural Details & Data Flow Explanation

1. **Userspace to Kernel Space Invocation:** Userspace programs interact with the device through the device node `/dev/lab2`. The VFS translates POSIX system calls (`open`, `read`, `write`, `close`) to the driver's registered `file_operations` table (`lab2_fops`) using Major number 240.
2. **Safe Memory Copy Flow:** During a `write()` operation, user data cannot be accessed directly via memory pointers. The driver executes `copy_from_user()` to validate memory boundaries and safely import bytes into the kernel's `device_buffer`. Conversely, `read()` transfers payload data back to userspace using `copy_to_user()`.
3. **Mutual Exclusion (Mutex):** Critical sections modifying `device_buffer`, `buffer_len`, and `open_count` are guarded by `DEFINE_MUTEX(lab2_mutex)` to guarantee data consistency in multi-threaded and SMP environments.
4. **Dual Diagnostic Interfaces:**
 - **procfs (`/proc/lab2_info`):** Implemented via the `seq_file` API and `single_open()` to output formatted multi-line runtime statistics for operator debugging.
 - **sysfs (`/sys/class/lab2_class/lab2/`):** Implemented using the `DEVICE_ATTR_RO` macro on the Linux `kobject` model, exporting discrete, atomic variables (`buffer_len`, `open_count`, `last_data`) according to the kernel standard of one value per file.

SECTION 3: IMPLEMENTATION LOGS & PRACTICAL RESULTS

3.1. Part A – Character Device Driver Implementation

3.1.1. Module Compilation and Target Binary Verification

The kernel module `lab2_driver.ko` was cross-compiled against the Linux 5.15.0 kernel tree using the GNU ARM toolchain.

```
baobao@XPS-13-9300:~/embedded_lab1/kernel/linux-5.15$ cd ~/embedded_lab2/driver
make clean && make
make -C /home/bao/embedded_lab1/kernel/linux-5.15 M=/home/bao/embedded_lab2/driver ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- clean
CLEAN /home/bao/embedded_lab2/driver/Module.symvers
make -C /home/bao/embedded_lab1/kernel/linux-5.15 M=/home/bao/embedded_lab2/driver ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules
CC [M] /home/bao/embedded_lab2/driver/lab2_driver.o
MODPOST /home/bao/embedded_lab2/driver/Module.symvers
CC [M] /home/bao/embedded_lab2/driver/lab2_driver.mod.o
LD [M] /home/bao/embedded_lab2/driver/lab2_driver.ko
baobao@XPS-13-9300:~/embedded_lab2/driver$ ls -lh lab2_driver.ko
file lab2_driver.ko
modinfo lab2_driver.ko | grep vermagic
-rw-r--r-- 1 bao bao 140K Sep 22 07:24 lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=dd5a7e33b62e0480fe53b91d8bbeb94e926a262d, with debug_info, not stripped
vermagic: 5.15.0 SMP mod_unload ARMv7 p2v8
baobao@XPS-13-9300:~/embedded_lab2/driver$
```

Figure 3.1: Module compilation, ARM ELF 32-bit validation, and vermagic matching check.

- **Detailed Observations:**

- The file utility confirms that `lab2_driver.ko` is an `ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV)` binary, verifying proper cross-compilation for Cortex-A9.
- Inspection using `modinfo lab2_driver.ko | grep vermagic` yielded: `5.15.0 SMP mod_unload ARMv7 p2v8`. This matches the target kernel running inside QEMU, preventing module loader rejection.

3.1.2. Device Node Registration and Read/Write Functionality Testing

The driver was tested using the automated test suite script `/usr/bin/test_driver.sh` inside the QEMU target system.

```

baob@bao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs — qemu-system-arm -M vxpress-a9 -cpu cortex-a9 -m 512M -smp 2 -nographic -kernel /home/bao/embedded_lab1...
~/embedded_lab2/rootfs/initramfs
modu [3] Checking/Creating device node...
crw-rw-rw- 1 root 0 240, 0 Sep 22 01:06 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!]
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1]
]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
Message_2
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
Message_3
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
=== Test PASSED ===
/ #

baob@bao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs — qemu-system-arm -M vxpress-a9 -cpu cortex-a9 -m 512M -smp 2 -nographic -kernel /home/bao/embedded_lab1...
~/embedded_lab2/rootfs/initramfs
modu lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
=====
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 9
Last data : [Embedded Linux Lab2 Test Data]
]
--- sysfs attributes ---
buffer_len : 30
open_count : 9
last_data : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
/ # dmesg | grep lab2
lab2_driver: Loading out-of-tree module taints kernel.
lab2_driver: Initializing module
lab2_driver: Loaded, major=240
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!]
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
lab2_driver: device closed
/ #

```

Figure 3.2: Execution output of test_driver.sh verifying device registration and read/write cycle.

- **Detailed Observations:**

- The module loaded into kernel memory with Major number 240.

- The device special file `/dev/lab2` was successfully verified (`crw-rw-rw- 1 root 0 240, 0 /dev/lab2`).
- String transmission test: Writing "Hello from userspace, LAB-02!" into `/dev/lab2` and reading it back returned the exact string intact.
- Sequential read/write loops (`Message_1`, `Message_2`, `Message_3`) executed without deadlock or buffer corruption.
- Kernel messages verified via `dmesg | grep lab2` confirmed sequential execution of `lab2_open()`, `lab2_write()`, `lab2_read()`, and `lab2_release()`.

3.2. Part B – Virtual Filesystem Communication (procfs & sysfs)

The `/proc` and `/sys` interfaces were validated using `/usr/bin/test_procfs.sh`.

```

bao@bao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs — qemu-system-arm -M vxpress-a9 -cpu cortex-a9 -m 512M -smp 2 -nographic -kernel /home/bao/embedded_lab1...
modu ]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
Message_3
lab2_driver: device closed
[7] Kernel messages (dmesg):
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device closed
=== Test PASSED ===
/ # test_procfs.sh
=== test procfs & sysfs ===
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 10 bytes
Open count    : 8
Last data     : [Message_3]
]
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 30 bytes
Open count    : 9
Last data     : [Embedded Linux Lab2 Test Data]
]
--- sysfs attributes ---
buffer_len : 30
open_count : 9
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
/ #

```

Figure 3.3: Dynamic runtime metric verification via `/proc/lab2_info` and `sysfs` attribute files.

● Detailed Observations:

- **procfs Dynamic Updates:** Prior to writing, `/proc/lab2_info` displayed `Data length: 10 bytes` and `Last data: [Message_3]`. After writing `"Embedded Linux Lab2 Test Data"`, reading the procfs node immediately reflected `Data length: 30 bytes`, `Open count: 9`, and `Last data: [Embedded Linux Lab2 Test Data]`, proving dynamic tracking.
- **sysfs Individual Attribute Nodes:** Querying the sysfs path `/sys/class/lab2_class/lab2/` yielded:
 - `buffer_len: 30`
 - `open_count: 9`
 - `last_data: Embedded Linux Lab2 Test Data`
- This confirms that attribute group creation via `sysfs_create_group()` is operating correctly.

3.3. Part C – MTD Subsystem & JFFS2 File System

3.3.1. NAND Flash Hardware Simulation (nandsim)

NAND flash memory was simulated on the host system using the `nandsim` kernel module configured for a 32MB STMicroelectronics NAND profile.

```
baob@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ cat /proc/mtd
dev: size erasesize name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"
baob@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ sudo mtdinfo /dev/mtd1
mtd1
Name: NAND simulator partition 0
Type: nand
Eraseblock size: 16384 bytes, 16.0 KiB
Amount of eraseblocks: 2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size: 256 bytes
OOB size: 16 bytes
Character device major/minor: 90:2
Bad blocks are allowed: true
Device is writable: true
baob@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ cat /proc/mtd
dev: size erasesize name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"
baob@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$
```

Figure 3.4: Host MTD partition table and mtdinfo parameters for simulated NAND flash.

● Detailed Observations:

- Examination of `/proc/mtd` revealed that `/dev/mtd0` was occupied by the physical motherboard BIOS NOR flash (`Name: "BIOS"`, 4KB erase size).

- The simulated NAND partition was correctly identified as `/dev/mtd1` (Name: "NAND simulator partition 0", Size: 32MB, Eraseblock size: 16.0 KiB / `0x4000`, Page size: 512 bytes). All subsequent operations targeted `/dev/mtd1` to preserve host integrity.

3.3.2. Erasing, Formatting, Mounting, and Persistence Verification

The raw flash was prepared, populated with initial data, and tested for persistence.

```

modu mtd1: 02000000 00004000 "NAND simulator partition 0"
bao@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ # 1. Xóa toàn bộ phân vùng mtd1
sudo flash_erase /dev/mtd1 0 0

# 2. Tạo thư mục chứa dữ liệu mẫu
mkdir -p /tmp/jffs2_data/logs
echo 'Hello from JFFS2!' > /tmp/jffs2_data/hello.txt
echo 'Embedded Linux Lab2' > /tmp/jffs2_data/info.txt
date > /tmp/jffs2_data/logs/boot.log

# 3. Đóng gói thành file JFFS2 image (eraseblock 16KB = 0x4000)
sudo mkfs.jffs2 --root=/tmp/jffs2_data --eraseblock=0x4000 --output=/tmp/rootfs.jffs2 --pad --no-cleanmarkers

# 4. Ghi image vào phân vùng mtd1
sudo nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2
Erasing 32768 Kibyte @ 0 -- 100 % complete
Writing data to block 0 at offset 0x0
Written 30 blocks containing only 0xff bytes
Those block may be incorrectly treated as empty!
bao@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ sudo mkdir -p /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs2

ls -la /mnt/jffs2/
cat /mnt/jffs2/hello.txt
cat /mnt/jffs2/info.txt
cat /mnt/jffs2/logs/boot.log
total 4
drwxr-xr-x 4 root root  0 Jan  1  1970 .
drwxr-xr-x 3 root root 4096 Sep 22 08:23 ..
-rw-rw-r-- 1 bao bao  18 Sep 22 08:22 hello.txt
-rw-rw-r-- 1 bao bao  20 Sep 22 08:22 info.txt
drwxrwxr-x 2 bao bao  0 Sep 22 08:22 logs
Hello from JFFS2!
Embedded Linux Lab2
Tue Sep 22 08:22:06 AM +07 2026
bao@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$ # Ghi thêm dữ liệu mới
sudo sh -c 'echo "Written at $(date)" >> /mnt/jffs2/logs/boot.log'
sudo sh -c 'echo "New runtime file" > /mnt/jffs2/runtime.txt'

# Unmount và mount lại
sudo umount /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs2

# Đọc lại để kiểm tra
cat /mnt/jffs2/logs/boot.log
cat /mnt/jffs2/runtime.txt
Tue Sep 22 08:22:06 AM +07 2026
Written at Tue Sep 22 08:23:48 AM +07 2026
New runtime file
bao@bao-XPS-13-9300:~/embedded_lab2/rootfs/initramfs$

```

Figure 3.5: Execution of `flash_erase`, `mkfs.jffs2`, `nandwrite`, mounting, and persistence verification.

- Detailed Observations:

- - `sudo flash_erase /dev/mtd1 0 0` completed 100%, resetting all eraseblocks to `0xFF`.
 -
 - An image was generated using `mkfs.jffs2 --eraseblock=0x4000 --pad --no-cleanmarkers` and programmed using `nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2`.
 -

- The partition was mounted at `/mnt/jffs2` via the MTD block emulation layer `/dev/mtdblock1`.
-
- **Persistence Test:** Initial files (`hello.txt`, `info.txt`, `boot.log`) were read successfully. An additional file `runtime.txt` was created, and data was appended to `boot.log`. The filesystem was unmounted (`umount /mnt/jffs2`) and remounted. Both files remained intact with all changes preserved, demonstrating complete filesystem persistence on raw flash.

○

3.3.3. Raw Flash Inspection (Magic Number Bitmask)

Raw NAND blocks were extracted to verify the internal JFFS2 node headers.

```

Usage: nanddump [OPTIONS] MTD-device
Dumps the contents of a nand mtd partition.

-h --help                Display this help and exit
-v --version             Output version information and exit
--bb=METHOD             Choose bad block handling method (see below).
-a --forcebinary         Force printing of binary data to tty
-c --canonicalprint      Print canonical Hex+ASCII dump
-f file                  Dump to file
-l length                Length
-n --noecc               Read without error correction
--omit00b                Omit 00B data (default)
-o --oob                 Dump OOB data
-p --prettyprint         Print nice (hexdump)
-q --quiet               Don't display progress and status messages
-s addr                  Start address
--skip-bad-blocks-to-start Skip bad blocks when seeking to the start address
-C --continuous          Continuous read up to a block of data at a time

--bb=METHOD, where METHOD can be 'padbad', 'dumppad', or 'skipbad':
  padbad: dump flash data, substituting 0xFF for any bad blocks
  dumppad: dump flash data, including any bad blocks
  skipbad: dump good data, completely skipping any bad blocks (default)
hexdump: all input file arguments failed
hexdump -C /tmp/dump.bin | head -15
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
00000000 85 19 01 e0 31 00 00 00 1d d9 82 42 01 00 00 00 |...1....B...|
00000010 00 00 00 00 02 00 00 00 3e d8 b1 6a 09 08 00 00 |.....>..j...|
00000020 ba 18 38 f0 15 da 69 f4 68 65 6c 6c 6f 2e 74 78 |..8...i.hello.tx|
00000030 74 ff ff ff 85 19 02 e0 56 00 00 00 09 64 e7 62 |t.....V....d.b|
00000040 02 00 00 00 01 00 00 00 b4 81 00 00 e0 03 e8 03 |.....|
00000050 12 00 00 00 3e d8 b1 6a 3e d8 b1 6a 3e d8 b1 6a |....>..j>..j>..|
00000060 00 00 00 00 12 00 00 00 12 00 00 00 00 00 00 00 |.....|
00000070 ee dd 4b b3 f0 63 8d 8f 48 65 6c 6c 6f 20 66 72 |..K...c..Hello fr|
00000080 6f 6d 20 4a 46 46 53 32 21 0a ff ff 85 19 01 e0 |om JFFS2!.....|
00000090 30 00 00 00 78 be 3e fa 01 00 00 00 01 00 00 00 |0...x>.....|
000000a0 03 00 00 00 3e d8 b1 6a 08 08 00 00 21 a2 49 7d |....>..j....!I|
000000b0 a9 de 79 72 69 6e 66 6f 2e 74 78 74 85 19 02 e0 |..yrinfo.txt....|
000000c0 58 00 00 00 3a 13 38 82 03 00 00 00 01 00 00 00 |X...:8.....|
000000d0 b4 81 00 00 e0 03 e8 03 14 00 00 00 3e d8 b1 6a |.....>..j>..|
000000e0 3e d8 b1 6a 3e d8 b1 6a 00 00 00 00 14 00 00 00 |>..j>..j.....|

```

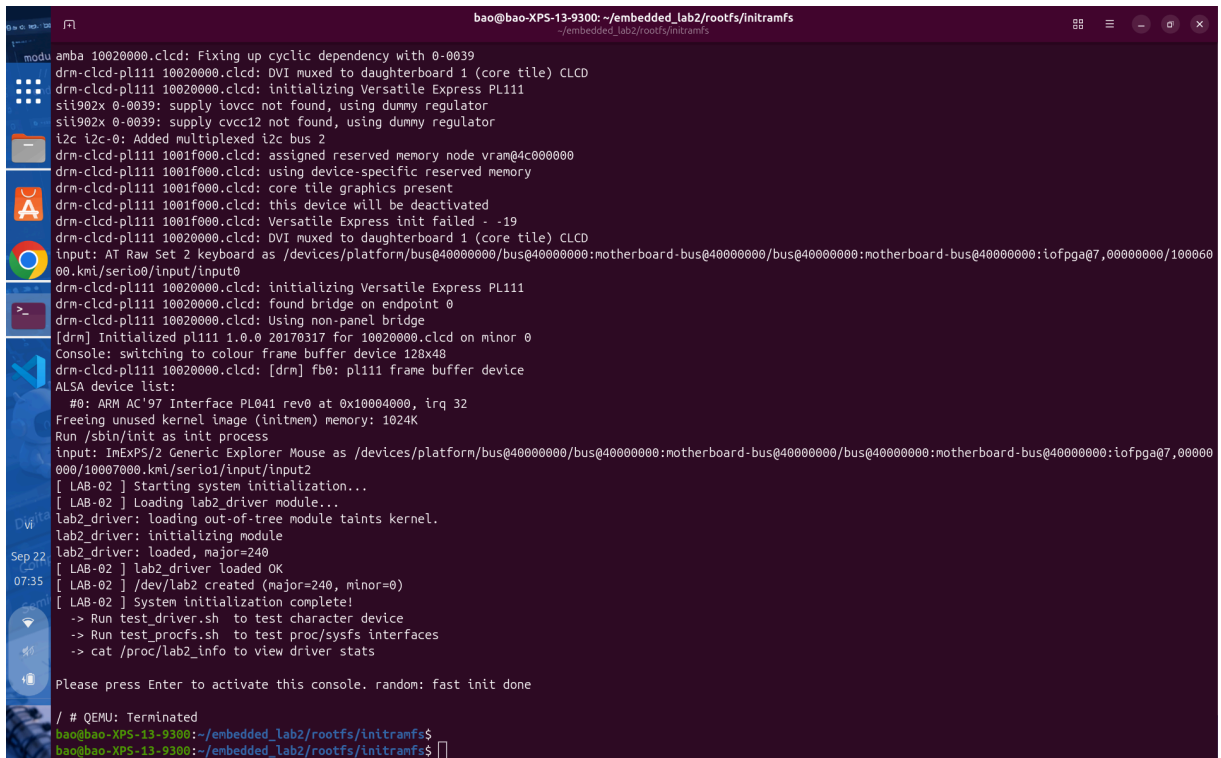
Figure 3.6: Hexdump of extracted NAND raw data verifying the JFFS2 magic number (0x85 0x19).

● Detailed Observations:

- Dumping the first 512 bytes with `nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd1` followed by `hexdump -C` revealed the leading magic bytes `85 19` (JFFS2_MAGIC_BITMASK) at offset `0x00000000`.
- The filenames `hello.txt` and `info.txt` are plainly visible in the raw node structures, confirming proper node assembly on flash.

3.4. Part D – BusyBox Init Auto-load Integration

The root filesystem was repackaged into `initramfs_lab2.cpio.gz` with an updated `/etc/init.d/rcS` startup script.



```
baobao@baobao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs
modu amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10006000.kmi/serio0/input/input0
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
#0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /sbin/init as init process
input: InEXPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: loaded, major=240
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
-> Run test_driver.sh to test character device
-> Run test_procfs.sh to test proc/sysfs interfaces
-> cat /proc/lab2_info to view driver stats
Please press Enter to activate this console. random: fast init done

/ # QEMU: Terminated
baobao@baobao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs$
baobao@baobao-XPS-13-9300: ~/embedded_lab2/rootfs/initramfs$
```

Figure 3.7: QEMU boot console showing automatic loading of `lab2_driver` during initialization.

● Detailed Observations:

- The kernel booted via QEMU with console redirected to `ttyAMA0`.
- The BusyBox init process executed the `::sysinit:/etc/init.d/rcS` action.
- The script detected `/lib/modules/5.15.0/lab2_driver.ko`, loaded it via `insmod`, registered Major 240, and automatically generated `/dev/lab2` before presenting the shell prompt.

SECTION 4: EMBEDDED FILE SYSTEMS COMPARISON & ANALYSIS

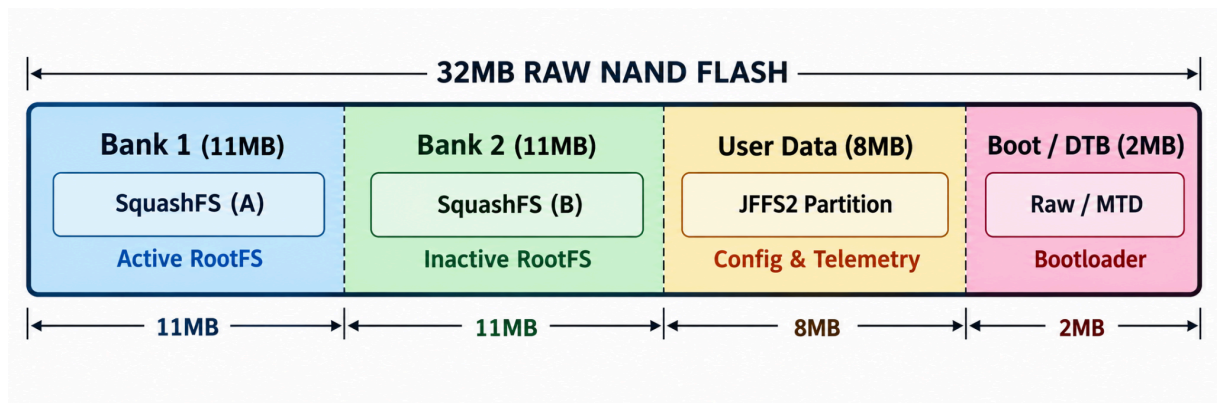
4.1. Comparative Analysis Table

Evaluation Metric	ext4	JFFS2	cramfs	SquashFS	UBIFS
Write Capability	Read / Write (Journaling)	Read / Write (Log-Structured)	Read-Only	Read-Only	Read / Write (on UBI volumes)
Wear-Leveling	None (Relies on hardware FTL)	Integrated (Software-managed)	N/A (Read-only media)	N/A (Read-only media)	Advanced (Handled by UBI layer)
Compression Support	None / Optional per-directory	Built-in (zlib, lzo, rttime)	Built-in (zlib)	High ratio (xz, zstd, gzip, lzo)	Built-in (LZO, zlib)
Mount / Boot Time	Instantaneous	Slow on large flash	Instantaneous	Fast	Very fast (Indexed metadata)

		(Full scan)			ata tree)
Target Storage Media	Block devices (eMMC, SD, SSD)	Small Raw NAND/N OR (<64MB)	Legacy ROM / Static flash	Block devices or Raw Flash	Large Raw NAND Flash (>64MB)

4.2. File System Recommendation for a 32MB NAND Flash IoT Device with OTA

For an embedded IoT device featuring **32MB Raw NAND Flash** requiring robust Over-The-Air (OTA) updates, an **asymmetric dual-bank partitioning architecture** is recommended:



1. Dual RootFS Partitions (Slot A and Slot B, ~11MB each) formatted with SquashFS:

- **Why SquashFS:** SquashFS provides the highest compression density available (saving up to 60-70% storage), which is vital on a constrained 32MB chip. Being read-only by design, it guarantees system integrity against sudden power disconnects or corruption during runtime.

- **OTA Resilience:** An A/B scheme allows the IoT client to write the incoming firmware update into the inactive slot in the background. After checksum verification, the bootloader toggles the active boot partition. If boot fails, the bootloader automatically rolls back to the known-good bank.

2. User Data & Configuration Partition (~8MB) formatted with JFFS2:

- **Why JFFS2:** This writable partition stores configuration files, cryptographic device tokens, and sensor telemetry logs. Because the allocated partition size is small (8MB), JFFS2's architectural drawback—scanning the entire filesystem at mount time—incurs negligible boot latency (a few milliseconds). It handles native flash wear-leveling and bad-block retirement without the steep RAM/metadata overhead of UBIFS (which typically requires a minimum of 32MB-64MB flash just for UBI metadata tables).

SECTION 5: IN-DEPTH TECHNICAL QUESTIONS ANALYSIS

Question 1: Kernel vs. User Space Memory & copy_to_user / copy_from_user Mechanism

- **Virtual Address Space Separation:** Modern architectures with MMUs isolate user applications into low virtual memory addresses (0x00000000 to 0xBFFFFFFF on 32-bit Linux), while kernel space occupies the higher address range (0xC0000000 to 0xFFFFFFFF). Each userspace process maintains its own translation page directory.
- **Why memcpy() Cannot Be Used:** The pointer passed into lab2_read() or lab2_write() (char __user *buf) references a virtual address belonging to the user process context. Dereferencing it directly with memcpy() in kernel context poses severe dangers:
 1. **Invalid Addresses & Exploits:** The pointer might point to unmapped space, read-only memory, or kernel memory space (allowing privilege escalation).

2. **Page Faults:** The user page might not be resident in physical RAM (paged out or lazily allocated). A standard `memcpy()` has no handling logic for page faults occurring within privileged CPU execution mode, triggering an immediate **Kernel Panic / Oops**.
- **Safety Mechanism:** `copy_to_user()` and `copy_from_user()` first invoke `access_ok()` to ensure the user buffer resides within the legal userspace boundary. They execute the transfer through assembly routines tied to the kernel's **Exception Fixup Table** (`.fixup / __ex_table`). If a page fault or invalid address is encountered, the kernel catches the MMU exception gracefully, aborts the copy, and returns `-EFAULT` to userspace without crashing the kernel.

Question 2: Mutex Synchronization and Race Condition Analysis

- **Necessity of Mutex Protection:** The driver shares a single static memory buffer (`device_buffer`) and state variables (`buffer_len`, `open_count`) across all callers. In a multi-tasking or multi-core SMP environment (such as our 2-core QEMU system), multiple user processes can invoke `write()` or `open()` simultaneously.
- **Concrete Race Condition Scenario without Mutex:**
 1. Process A calls `write("/dev/lab2", "AAAAA", 5)` and enters `lab2_write()`. It executes `copy_from_user()`, writing "AAAAA" into `device_buffer`.
 2. Before Process A executes `buffer_len = 5;`, the kernel preemption timer expires, causing a context switch to Process B.
 3. Process B calls `write("/dev/lab2", "BBBBBBBB", 8)`. It executes `copy_from_user()`, overwriting `device_buffer` with "BBBBBBBB", and updates `buffer_len = 8`.
 4. Execution switches back to Process A, which resumes at `buffer_len = 5;`.

5. **Consequence:** `device_buffer` contains "BBBBBBBB", but `buffer_len` indicates 5. Subsequent reads will truncate the string, or in reverse order, expose uninitialized memory. If data writes are interleaved byte-by-byte, memory corruption occurs. The `DEFINE_MUTEX(lab2_mutex)` construct serializes access, forcing Process B to sleep until Process A completes its transaction.

Question 3: Comparison of `procfs` (`/proc`) and `sysfs` (`/sys`) Interfaces

- **Core Distinction:**
 - `procfs` was designed to export process metadata (`task_struct`). Over time, it accumulated miscellaneous kernel statistics, resulting in an unorganized namespace.
 - `sysfs` was introduced in Linux 2.6 as an integral part of the **Unified Device Model** (`kobject` infrastructure), strictly mirroring physical buses, device classes, and driver hierarchies.
- **Kernel Policy:** The official kernel documentation deprecates adding general driver parameters to `/proc`. Instead, `sysfs` is required, governed by the strict **"one value per file"** rule. This allows userspace programs (such as `udev` and monitoring scripts) to parse values directly without parsing tabular strings.
- **Application in `lab2_driver`:**
 - `/proc/lab2_info`: Appropriate for `procfs` because it exports a multi-line, formatted aggregate summary (Major number, capacity, length, open counts, last data) intended for human operators via `seq_file`.
 - `/sys/class/lab2_class/lab2/`: Appropriate for `sysfs` because metrics are decomposed into individual attribute nodes (`buffer_len`, `open_count`, `last_data`) utilizing `DEVICE_ATTR_RO`, allowing automated tools to query specific parameters independently.

Question 4: Physical Characteristics of NAND Flash and JFFS2 Log-Structured Mechanism

- **Physical Constraints of NAND Flash:**
 - A NAND flash cell can only be programmed from binary **1** to **0** at the **Page** level (512B - 4KB).
 - Reversing bits from **0** back to **1** requires applying a high electrical potential across an entire **Eraseblock** (16KB - 256KB).
 - Therefore, flash memory does not support in-place overwrites. Writing updated data to an existing page requires erasing the whole block first.
- **Block Devices vs. Raw Flash:** Block devices (eMMC, SSDs, SD cards) integrate a microcontroller running a Flash Translation Layer (FTL) that emulates standard disk sectors. Traditional filesystems like **ext4** write to logical block addresses without managing physical flash geometry. Raw flash has no FTL, leaving block management to the OS.
- **JFFS2 Log-Structured Operation:** JFFS2 treats raw flash as an append-only log. When a file modification occurs, JFFS2 does not overwrite the old data; it writes a new data node containing the updated payload and a higher revision counter at the next available free flash offset. The previous node is marked as obsolete. A background **Garbage Collector (GC)** thread identifies blocks containing high fractions of obsolete nodes, copies the valid nodes to a fresh eraseblock, and executes **flash_erase** on the old block to prepare it for reuse, delivering integrated wear-leveling.

Question 5: BusyBox Init Resilience and rcS Script Enhancement

- **Boot Continuation on Module Failure:** BusyBox **/sbin/init** reads **/etc/inittab** and forks **/bin/sh** to execute **/etc/init.d/rcS** under the **sysinit** directive. Shell scripts run sequentially; if a command like **insmod** returns a non-zero exit code (failure), the shell prints an error message and continues to the next command unless configured with **set -e** or an

explicit exit trap. Thus, the system will still finish booting into the console.

- **Distinguishing Critical vs. Non-Critical Failures:** Critical errors (failure to mount `/dev`, missing essential storage drivers) must be trapped to prevent silent data corruption or panics. Non-critical driver failures should log warnings and let the system proceed.
- **Enhanced `rcS` Script Implementation:**

```
#!/bin/sh

# Enhanced /etc/init.d/rcS with error handling and fallback shell

mount -t proc none /proc

mount -t sysfs none /sys

mount -t devtmpfs none /dev 2>/dev/null || true

# Module configuration

MODULE_PATH="/lib/modules/5.15.0/lab2_driver.ko"

IS_CRITICAL=0 # Set to 1 if driver failure must halt initialization

echo '[ LAB-02 ] Loading lab2_driver module...'

if [ -f "$MODULE_PATH" ]; then
    insmod "$MODULE_PATH"
    ERR_CODE=$?
    if [ $ERR_CODE -eq 0 ]; then
        echo '[ LAB-02 ] lab2_driver loaded successfully'
        mknod /dev/lab2 c 240 0 2>/dev/null || true
        chmod 666 /dev/lab2
    else
        echo "[ LAB-02 ] ERROR: insmod failed with return code $ERR_CODE"
        if [ $IS_CRITICAL -eq 1 ]; then
```

```

    echo '[ LAB-02 ] FATAL: Critical driver failed. Launching recovery shell...'

    exec /bin/sh

else

    echo '[ LAB-02 ] WARNING: Proceeding with non-critical driver disabled'

fi

fi

else

    echo "[ LAB-02 ] WARNING: Module file $MODULE_PATH not found"

fi

echo '[ LAB-02 ] Initialization completed.'
```

SECTION 6: PRACTICAL DIFFICULTIES & SOLUTIONS

During the implementation of Lab 2, our team encountered and resolved two critical technical issues:

6.1. Issue 1: GNU Assembler Instruction Synchronization Barrier (`isb`) Error

- **Symptom:** During out-of-tree module compilation using GCC 15 (`arm-linux-gnueabi-hf-gcc 15.2.0`), the build halted with assembler errors:
-

```
/tmp/cc3N81XJ.s: Assembler messages:
```

```
/tmp/cc3N81XJ.s:583: Error: selected processor does not support `isb' in ARM mode
```

- **Root Cause:** The `isb` (Instruction Synchronization Barrier) instruction is native to the ARMv7-A architecture (Cortex-A9). Due to behavioral changes in GCC 15, the compiler failed to automatically propagate target sub-architecture flags to the assembler during standalone module compilation, defaulting to a legacy ARM instruction set (ARMv4T/ARMv5) where memory barrier instructions are invalid.
- **Resolution:** We updated the module's `Makefile` to explicitly inject the target architecture flag into the compilation pipeline:


```
ags-y += -march
```
- This forced the GNU Assembler to compile instructions under the ARMv7-A specification, resolving the error.

6.2. Issue 2: Missing ARM Module Linker Script (`module.lds`) and Symbol Table

- **Symptom:** Subsequent compilation failed during the final modpost stage with:
 - ```
make[3]: *** No rule to make target 'scripts/module.lds', needed by 'lab2_driver.ko'. Stop.
```

WARNING: Symbol version dump "Module.symvers" is missing.
    - **Root Cause:** In Lab 1, the kernel source tree was only compiled for the base monolithic kernel image (`zImage`). The architecture-specific linker script required for dynamic ARM modules (`scripts/module.lds`) and the exported kernel symbol definition index (`Module.symvers`) had not been prepared within the kernel tree.
    - 
    - **Resolution:** We navigated to `~/embedded_lab1/kernel/linux-5.15` and executed:
 

```
RCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules_prepare
```

```
make -j$(nproc) ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules
```
    - This generated `scripts/module.lds` and `Module.symvers`, allowing `lab2_driver.ko` to compile without unresolved symbols.
    -
- (Practical Note: During host MTD testing, `sudo mtdinfo /dev/mtd0` targeted the physical motherboard BIOS NOR flash of the Dell XPS host laptop. By inspecting `/proc/mtd`, our team correctly redirected commands to `/dev/mtd1`, preventing hardware corruption).